

Technische Universität Clausthal

Department of Informatics

Beyond Pairwise GUI Quality Judgments

Multi-Dimensional Evaluation with Modern Multimodal LLMs

Master's Thesis

Xinyang Xie

April 30, 2026

Contents

1	Introduction	1
2	Background and Related Work	2
2.1	Research Gap	2
3	Research Questions and Methodology	5
3.1	Research Questions	5
3.2	Overall Study Design	6
3.3	Track A: Screenshot-based Reproduction	7
3.4	Track B: Requirement-driven Generated Interfaces	8
3.4.1	Requirement Sources	8
3.4.2	Interface Generation	8
3.4.3	Rendering Pipeline	8
3.5	Static Evaluation Methodology	9
3.5.1	Model Selection	9
3.5.2	Prompting Conditions	9
3.5.3	Reliability Conditions	9
3.6	Dynamic Evaluation Methodology	9
3.6.1	Approach	9
3.6.2	Task Derivation	10
3.6.3	Failure Taxonomy	10
3.7	Metrics	11
3.8	Research Question – Method Map	11

Chapter 1

Introduction

This chapter will introduce the motivation, research context, and contribution of the thesis. It is intentionally kept as a short placeholder so that the following draft chapters keep their intended numbering while the full introduction is developed.

Chapter 2

Background and Related Work

2.1 Research Gap

The preceding sections have reviewed GUI quality assessment, LLM-generated interfaces, LLM-as-a-judge methodology, and multimodal web agents. Table 2.1 places the most relevant prior works along seven dimensions that together characterise a comprehensive GUI quality evaluation framework. Taken together, three observations motivate the present thesis.

No prior work applies modern multimodal LLMs as multi-dimensional GUI judges. UIClip [7] demonstrated that vision-language models can perform pairwise UI comparison, but it relies on models that predate the current generation of multimodal LLMs and limits evaluation to a binary preference. The LLM-as-a-judge literature [9, 3] has established systematic judge evaluation for text, yet this methodology has not been transferred to the UI domain.

Requirement fidelity and rubric-based assessment are largely absent from GUI evaluation. Design2Code [6] evaluates how faithfully generated code reproduces a visual reference, but does not assess richer quality dimensions such as visual hierarchy, information clarity, or perceived usability. The Designer Feedback study [8] found that simple pairwise rankings among evaluators agreed at only around 50%, suggesting that coarse binary judgements are insufficient for reliable GUI assessment. A structured rubric grounded in established usability theory [4, 1] can provide finer-grained and more reproducible quality signals.

Static visual judgements have never been validated against dynamic interaction outcomes. VisualWebArena [2] and WebArena [10] evaluate multimodal agents on goal-directed web tasks, but do not assess the GUI quality of the interfaces themselves. Whether a high static quality score predicts successful task completion by

a computer-use agent remains an open empirical question. Answering this question requires a benchmark that keeps generated interfaces executable so that static and dynamic evaluations can be performed on the same set of items.

The present thesis addresses all three gaps simultaneously by combining a two-track benchmark, a rubric-based human annotation protocol, a systematic comparison of modern multimodal LLM judges with bias and reliability analysis, and a dynamic evaluation module that links static scores to interaction outcomes.

Table 2.1: Comparison of related work across seven evaluation dimensions. ✓ = fully addressed; △ = partially addressed; × = not addressed; — = not applicable to this line of work.

Work	Multi-dimensional rubric	Modern multimodal LLM judge	Bias and reliability analysis	Human reference labels	Requirement fidelity	Dynamic interaction eval	Benchmark artifact
UIClip [7]	×	×	×	✓	×	×	×
Designer Feedback [8]	×	×	×	✓	△	×	×
Design2Code [6]	△	✓	×	✓	✓	×	✓
MT-Bench / Chatbot Arena [9]	△	✓	△	✓	—	—	△
LLM-as-a-Judge survey [3]	✓	✓	✓	—	—	—	×
Position Bias Study [5]	×	✓	✓	✓	—	—	×
WebArena [10]	×	△	×	✓	△	✓	✓
VisualWebArena [2]	×	✓	×	✓	△	✓	✓
This thesis	✓	✓	✓	✓	✓	✓	✓

Chapter 3

Research Questions and Methodology

This chapter translates the motivation established in Chapter 2 into four precise research questions and describes the experimental design through which each question is answered. Section 3.1 states the research questions. Section 3.2 outlines the overall study design. Sections 3.3–3.6 detail the two dataset tracks and the static and dynamic evaluation methodologies. Section 3.7 defines all evaluation metrics. Section 3.8 maps each research question to the corresponding dataset, tasks, metrics, and expected outputs.

3.1 Research Questions

The central concern of this thesis is whether modern multimodal large language models can serve as reliable judges of GUI quality, and whether the quality signals they produce carry meaning beyond the visual surface of a screenshot. Four research questions operationalise this concern.

RQ1 – Modern models as pairwise GUI judges.

How well do modern multimodal large language models align with human pairwise judgements of GUI quality, and how does their performance compare to the models used in UIClip [7]?

This question targets the reproduction aspect of the study (Track A, Section 3.3). It asks whether progress in multimodal modelling has translated into better GUI preference judgements, using the UIClip dataset as an anchor that allows direct numerical comparison with published results.

RQ2 – Multi-dimensional rubric alignment.

When evaluation moves from binary pairwise comparison to multi-dimensional rubric scoring, how well do modern multimodal LLMs align with human ratings per dimension, and which rubric dimensions are most and least reliably judged?

This question applies to both Track A and Track B (Section 3.4). It examines whether LLMs can assign human-aligned scores along specific quality axes – visual structure, information clarity, requirement fidelity, and interaction quality – rather than returning only a binary preference.

RQ3 – Judging strategies and reliability.

Among the tested judging strategies – zero-shot prompting, rubric-guided prompting, order-swapped pairwise comparisons, and repeated sampling – which configurations achieve the best trade-off between alignment with human labels and API cost?

This question is answered across both tracks. It draws on the LLM-as-a-judge literature [9, 3, 5] to select strategies known to affect judge reliability, and applies them systematically to the GUI evaluation setting.

RQ4 – Static judgements and dynamic interaction outcomes.

For the same set of executable Track B interfaces, do static LLM rubric scores predict the success rate of LLM-derived interaction plans on derived tasks, and which rubric dimensions are most predictive of dynamic outcomes?

This is the core novel question of the thesis. Prior work has evaluated GUI quality from screenshots or evaluated agents on web tasks, but the two lines of research have not been connected on the same set of interfaces. RQ4 addresses this gap by pairing a static rubric score vector with a dynamic plan-success rate for every Track B item and measuring their correlation.

3.2 Overall Study Design

Figure 3.1 illustrates the end-to-end study design. The evaluation is organised around two complementary dataset tracks.

Track A consists of GUI screenshot pairs drawn from the UIClip dataset. Its primary function is reproduction and anchoring: by running modern multimodal LLMs on the same pairwise task that UIClip used, Track A enables a direct numerical comparison that establishes whether contemporary models represent a measurable advance. Track A items undergo static evaluation only.

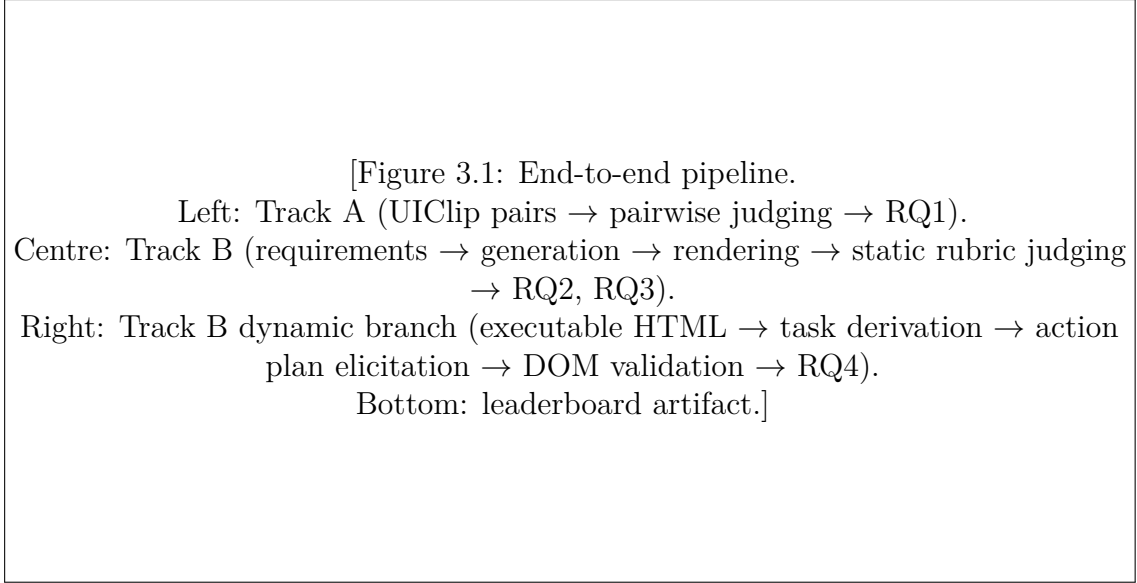


Figure 3.1: End-to-end evaluation pipeline. Track A supports reproduction (RQ1). Track B supports rubric-based assessment (RQ2, RQ3) and dynamic plan validation (RQ4).

Track B consists of graphical user interfaces generated from natural language requirements. Every Track B interface is kept in executable form so that it can be assessed both statically – through rubric-based scoring – and dynamically, through LLM-derived interaction plans that are validated against the rendered DOM. The key design principle is that the same interface receives both a static rubric score vector and a dynamic plan-success measurement, forming the paired data needed to answer RQ4.

Human annotations provide the reference labels for both tracks. Track A reuses the preference labels published with the UIClip dataset, so no new annotation is required for pairwise tasks on that track. Track B requires fresh human rubric scores, collected according to the annotation protocol described in the benchmark chapter.

All experiments are documented with exact model identifiers, API version dates, prompt text, random seeds, and output files to support reproducibility.

3.3 Track A: Screenshot-based Reproduction

Track A draws on the publicly available UIClip dataset [7], which contains pairs of GUI screenshots each labelled with a human preference indicating which interface is visually superior. A random sample of 50 to 100 pairs is selected, stratified to cover the range of quality contrasts present in the original dataset.

No new human annotation is collected for Track A. The original UIClip preference labels serve as ground-truth reference, and agreement between each judge model and those labels constitutes the alignment measure. This design makes Track A lightweight and directly comparable to the UIClip publication while still allowing the main novel

contributions – multi-dimensional rubric assessment and dynamic evaluation – to be concentrated in Track B.

3.4 Track B: Requirement-driven Generated Interfaces

3.4.1 Requirement Sources

Track B requirements are collected from two sources. The primary source is the Design2Code-HARD subset [6], which contains 80 challenging real-world webpages. For each selected page, the corresponding HTML reference is used to derive a concise natural language requirement that describes the intended layout, content, and functionality without specifying implementation details. Approximately 50 to 60 pages are selected after filtering out items whose interactive functionality cannot be meaningfully assessed through DOM inspection alone.

A secondary set of 5 to 10 requirements is written manually to cover scenarios underrepresented in Design2Code-HARD, specifically form-heavy interfaces and simplified mobile-style layouts. These types support more tractable dynamic task derivation (Section 3.6) and reduce reliance on a single source.

3.4.2 Interface Generation

Each requirement is submitted to two or three generator models, producing a set of generated HTML implementations per requirement. The generator models are drawn from the same pool as the judge models (Section 3.5) to allow analysis of whether a model judges its own outputs differently from those of other generators.

3.4.3 Rendering Pipeline

Generated HTML files are rendered using a headless browser (Playwright) at a fixed viewport resolution. For each item, the pipeline saves a PNG screenshot and a serialised DOM snapshot. Items whose rendering produces a blank page or a JavaScript console error that prevents visible content from loading are excluded and replaced. The rendered screenshot and the DOM snapshot together constitute the input to both the static and dynamic evaluation modules.

3.5 Static Evaluation Methodology

3.5.1 Model Selection

Four judge models are tested to span a range of capability levels. Two closed-source multimodal models represent the current state of the art; one open-weight multimodal model is included to allow reproducibility without API access; and one smaller or older multimodal model serves as a weaker baseline that makes the relationship between model capability and judge quality visible. Exact model identifiers and access dates are recorded in the reproducibility appendix.

3.5.2 Prompting Conditions

Two prompting conditions are applied to every model.

Zero-shot prompting presents the model with the interface screenshot and a minimal instruction to indicate which of two interfaces is better (pairwise) or to score the interface on the rubric dimensions. This condition establishes a baseline.

Rubric-guided prompting supplies the full rubric with dimension definitions and per-level score anchors, and requires the model to return a structured JSON response containing one integer score per dimension and a one-sentence rationale. This condition tests whether explicit guidance improves alignment with human ratings.

3.5.3 Reliability Conditions

Two reliability conditions are applied on top of the prompting conditions.

Order-swapped pairwise comparison repeats each pairwise judgement with the two interface screenshots presented in the reverse order [5]. If a model labels interface A as better when it appears first but interface B as better when it appears first, the judgement is classified as position-biased. The position-bias rate is computed per model and per prompting condition.

Repeated sampling submits each prompt three times with a non-zero sampling temperature. The self-consistency rate measures the proportion of items on which all three runs produce the same judgement.

3.6 Dynamic Evaluation Methodology

3.6.1 Approach

A computer-use agent that executes browser actions introduces two confounded sources of noise: failures caused by the interface itself and failures caused by the agent’s own

planning or grounding limitations. Disentangling these sources is difficult and adds substantial engineering overhead.

Instead, this thesis uses *action plan elicitation*: a judge model is given the rendered screenshot, the DOM snapshot, and a short task description derived from the original requirement, and is asked to produce an ordered action plan – a JSON list of steps, each specifying an action type, a CSS or ARIA target selector, and an optional value. A deterministic validation script then checks whether each step in the plan is feasible: whether the target element exists in the DOM, whether the selector resolves to a unique element, and whether a form-submission step reaches a plausible end state. This design isolates interface quality from agent execution variability and produces a binary feasibility verdict for each action step, giving a plan-success rate per item.

Full computer-use agent execution – where an agent autonomously interacts with the live rendered page – is left as a future extension.

3.6.2 Task Derivation

One to two tasks are derived from each Track B requirement. A task is a single imperative sentence with an unambiguous success criterion that can be checked by inspecting the DOM or a screenshot of the expected end state. Example tasks include filling and submitting a contact form, selecting an item from a navigation menu, or entering a search query. Tasks are written by the author and cross-checked by one independent reviewer to confirm clarity and feasibility.

3.6.3 Failure Taxonomy

Action plan failures are classified into four categories to support interpretable analysis of RQ4.

F1 – Interface failure: the target element is absent from the DOM, the layout prevents meaningful interaction, or a required interactive component is not implemented.

F2 – Plan failure: the element exists but the model’s selector does not resolve correctly, or the action sequence does not reach the target state despite the interface being technically functional.

F3 – Ambiguity: the task description admits multiple valid interpretations and the model’s plan addresses a different valid interpretation than the one used by the validator.

F4 – Tool failure: the validation script itself produces an error unrelated to the interface or the plan.

Cases in category F3 and F4 are reviewed manually and excluded from the RQ4 correlation analysis if they cannot be resolved.

3.7 Metrics

Human-model agreement (pairwise). Cohen’s κ measures agreement between a judge model’s pairwise preference and the human majority label, correcting for chance.

Human-model correlation (rubric). Spearman’s ρ measures rank correlation between model rubric scores and human rubric scores, computed per rubric dimension and reported both individually and averaged.

Position-bias rate. The proportion of pairwise items on which the model’s preference reverses between the original and order-swapped presentation.

Self-consistency rate. The proportion of items on which all three repeated-sampling runs produce the same judgement or score.

Plan-success rate. The proportion of action steps in a plan that pass all three feasibility checks (existence, unique resolution, state transition). Averaged across steps within an item to obtain an item-level score.

Static-dynamic correlation. Spearman’s ρ between the item-level static rubric score (aggregate and per-dimension) and the item-level plan-success rate, computed across all Track B items.

API cost. Total token cost in USD per model per condition, computed from provider pricing at the time of the experiment and used to characterise the alignment-cost trade-off addressed in RQ3.

3.8 Research Question – Method Map

Table 3.1 summarises the relationship between each research question, the dataset track and task type used to answer it, the primary metrics, and the results section that reports the findings.

Table 3.1: Research question – method mapping.

RQ	Question (short)	Track	Task type		Primary metrics
1	Modern multimodal LLMs vs UIClip in pairwise GUI comparison	Track A	Pairwise judging		Cohen’s κ ; accuracy vs UIClip labels
2	Human-model alignment in multi-dimensional rubric scoring	Track A + B	Rubric	scoring	Spearman ρ per dimension
3	Best judging strategy for alignment-cost trade-off	Track A + B	Pairwise rubric	+	κ ; ρ ; position-bias rate; self-consistency rate; API cost
4	Static rubric scores as predictors of dynamic task success	Track B	Rubric + action plan validation	+	Spearman ρ (static-dynamic); plan-success rate; failure categories

Bibliography

- [1] International Organization for Standardization. ISO 9241-11:2018 — ergonomics of human-system interaction: Usability: Definitions and concepts. International Standard 9241-11, ISO, 2018.
- [2] Jing Yu Koh, Robert Lo, Lawrence Yunliang Chen, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proc. ACL*, 2024.
- [3] Dawei Li, Bohan Jiang, Liangjie Huang, et al. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proc. EMNLP*, 2025.
- [4] Jakob Nielsen and Rolf Molich. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 249–256. ACM, 1990.
- [5] Lin Shi, Chiyu Ma, Wenhua Liang, et al. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Proc. IJCNLP-AAACL*, 2025.
- [6] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, Hannes Hapke, Caiming Xiong, and Diyi Yang. Design2Code: How far are we from automating front-end engineering? In *Proc. ACL*, 2024.
- [7] Jason Wu, Yi-Hao Peng, Xin Yue Amanda Li, et al. UIClip: A data-driven model for assessing user interface design. In *Proc. ACM UIST*, 2024.
- [8] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, et al. Improving user interface generation models from designer feedback. In *Proc. CHI*, 2026.
- [9] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2023.
- [10] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *Proc. ICLR*, 2024.